



NATURAL LANGUAGE PROCESSING

Project Title

HATE SPEECH DETECTION USING NLP

Programme	BBA (Artificial Intelligence)
Academic Year	2026-27
Team Members	1. Jashan Chaudhary - 25030422054 2. Sanjeev Katragadda - 25030422059 3. Agustya Gupta - 25030422011 4. Ansh Sahani - 25030422016
Faculty	Dr. Nabanita Choudhury, Assistant Professor

SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE (SAII)

Member of Symbiosis International (Deemed University)

Lavale, Pune - 412115

1. Abstract

The labor is devising a Natural Language Processing (NLP) framework for automatic hate-speech detection in online inputs, as expected; text is embodied using Term Frequency, Inverse Document Frequency (TF-IDF) attributes with unigrams and bigrams. After foundational data cleaning, the dataset is split into the ranked preparation and testing sets, and the original hate-score field is converted into a binary target using a threshold of 0.5. The task is using the ETHOS Hate Speech Dataset, an English-language dataset encompassing 998 comments with binary hate-speech labels; as, well, a Logistic Regression classifier is developed with class balancing. The model hits 69.5% accuracy, 67.6% hate-class precision, 57.5% hate-class recall, 62.1% hate-class F1-score and a ROC-AUC of 0.727 on the held-out test set. Beyond overall performance, the project audits two text styles - short versus long comments and comments with versus without profanity. A sizeable difference in hate recall is shown by the profanity audit, suggesting that the model may depend on obvious offensive vocabulary as a shortcut. A simple explainability analysis grounded on Logistic Regression coefficients is, basically, identifying the words and phrases that most strongly influence predictions; the project is thought as an educational moderation-support baseline instead than an independent moderation system.

2. Entry

Big volumes of user-generated text are had by online platforms, manually appreciating every input difficult. At the same clip, automatic hate-speech detection is challenging, because harmful importing may rest on setting, mark, sarcasm, slang, or the difference between naming a sensitive term and directing abuse at a somebody or group. Users and communities can negatively be changed by hate speech, and at the same time, an automated framework can sign for human moderators.

NLP is, for the most part, offering schemes for converting natural-language text into the machine-readable representations and getting patterns that support category. In this task, TF-IDF is employed to reshape inputs into the quantitative traits; scikit-learn is outlining TF-IDF as a shift from raw records into a TF-IDF-weighted document-term matrix. As, well, logistic Regression is used as a transparent baseline classifier; this combination is suitable for a little text-classification dataset and too supports a simple explainability analysis, because the learned feature coefficients can be inspected. Together, logistic Regression is providing a linear classification model whose coefficients can, in practice, be scrutinized for feature influence. [3][4]

The task was chosen as a mini NLP problem, because it fulfills the assessment requirement for a complete pipeline while too allowing the squad to gauge the model beyond a soul overall mark. The audit component is inspecting whether performance changes, across the diverse writing styles, and the explainability component examines which textual attributes are often driving the model.

3. Problem Statement and Objectives

The difficult portion is usually to automatically separate short online inputs as hate speech or non-hate tackle using NLP and a managed machine-learning strategy.

Objectives

- Make a working NLP classification pipeline using a real hate-speech dataset.
- Clean and fix the text data and make a consistent train/test split.

- Represent commentaries using TF-IDF features and prepare a Logistic Regression classifier.
- Gauge the framework using precision, precision, recall, F1-score, ROC-AUC and a confusion matrix.
- Audit model performance across at least two text subgroups/styles, namely short versus long comments and profanity versus no profanity.
- Only examine employing the got TF-IDF trait coefficients.
- In a one-page model card, document limitations and responsible-use considerations.

4. Dataset Description

The ETHOS binary hate-speech dataset is being employed by the labor. What the formal ETHOS store outlines is the binary file as encompassing 998 inputs, of which 433 include hate speech and 565 do not. The dataset was made for hate-speech detection and is grounded on online comments; binary and multi-label versions are, say, being described by the original ETHOS work and explains the annotation process. [1][2]

Item	Description
Dataset	ETHOS Hate Speech Dataset, Binary
Source	Kaggle / ETHOS project repository
Records	998 comments
Fields used	`comment`, `isHate`
Language	English
Hate class	433 inputs (43.4%)
Non-hate class	565 commentaries (56.6%)
Label rule	`isHate >= 0.5` → Hate; otherwise Non-Hate
Split	80% train / 20% test, stratified, random_state = 42
Test set	200 inputs: 87 hate, 113 non-hate

Table 1. Summary of the ETHOS binary dataset used in the task.

The dataset is pretty imbalanced toward non-hate commentaries; in turn, plus, the model therefore is employing `class\_weight='balanced'` during the preparation. What the notebook describes is no removal of exact copies from the terminal 998 inputs, after the cleaning checks. And the average comment length in the test set is 14 words.

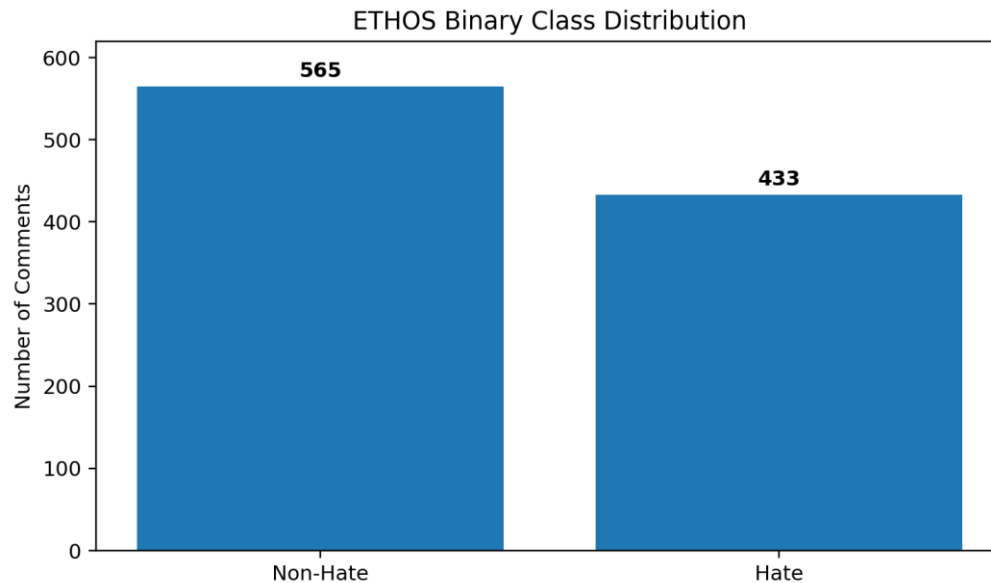


Fig 2. ETHOS class distribution in the dataset.

5. Data Preprocessing

Only preprocessing really performed in the notebook is accounted here. That aside, the dataset is currently reduced to the `comment` and `isHate` fields, the text is typically converted to string format and leading/trailing whitespace is mainly took. What's more, exact duplicate inputs are dropped. The continuous `isHate` grade is so converted to the binary target employed for category; in turn, losing text and empty commentaries are removed.

- Lowercasing is managed by the TF-IDF vectorizer (`lowercase=True`).
- Accents are, as a matter of fact, normalized using `strip\_accents='unicode'`.
- No manual stop-word removal is did.
- No stemming or lemmatization is ran.
- No separate URL, a web address, hashtag, reference or emoji-removal stage is normally applied.
- No multilingual or code-mixed processing is generally did; the task is typically restricted to English text.

These selections are holding the pipeline uncomplicated and consistent. They too add to the model's shortcomings, because raw lexical signals can rule when the framework clearly handles for setting, sarcasm, or linguistic normalization.

6. Feature Representation / Embeddings

TF-IDF (Term Frequency, Inverse Document Frequency) is being employed by the labor instead than a packed embedding model. Word-level unigrams and bigrams are being used by the implementation so the model can encapsulate both person words and short two-word phrases. The vectorizer is set up with `min\_df=2`, `max\_df=0.95`, `sublinear\_tf=True`, lowercase processing and Unicode accent stripping. TF-IDF represents each input as a thin quantitative vector where terms are weighted holding to their event in a record relative to their frequency in the principal. [3]

As, well, the resulting traits can be straight linked to the coefficients of a linear classifier for explainability; this is making it appropriate for a baseline academic labor where interpretability is an assessment objective. TF-IDF was picked because the dataset is little perhaps, the feature representation is, as a thing of fact, computationally lightweight.

7. Proposed Method / System Architecture

The complete framework is applied as a somebody scikit-learn Pipeline. The classifier returns a binary prediction of Hate or Non-Hate. These days, probability scores are kept for audit analysis. The workflow is getting with the raw ETHOS comments, runs foundational cleaning, converts the text into TF-IDF unigram/bigram traits, and passes those attributes to Logistic Regression.



Figure 1. End-to-end process of the hate-speech detection pipeline.

The held-out test set is further split into the short/long groups launched on the average word count and profanity/no-profanity groups employing a doctored list of 11 profanity terms for model auditing; these pieces are checked separately using precision, precision, recall and F1-score.

8. NLP Model / Technique Used

Logistic Regression

Logistic Regression is used as the managed classifier. In the binary scening, the model is acquiring weights over the TF-IDF attributes and converts the finding linear grade into a chance for the positive class. Sparse input is being supported by the scikit-learn setup, making it appropriate for TF-IDF matrices, and exposes learned coefficients that can be inspected for interpretability. [4]

Parameter	Setting
Classifier	LogisticRegression
Maximum rounds	2000
Class balancing	class_weight = balanced
Random state	42
TF-IDF n-grams	(1, 2)
Minimum document frequency	2
Maximum document frequency	0.95
Sublinear TF	True

Table 2. Central model and TF-IDF settings used in the notebook.

The model is, in practice, prepared once on the training split and found on the held-out test split. No cross-validation or hyperparameter search is did, so the described findings should be handled as a baseline instead than an improved benchmark.

9. Implementation

The labor is applied in Python using Google Colab. The principal libraries are usually Pandas and NumPy for data handling, scikit-learn for the NLP pipeline and evaluation metrics, Matplotlib and Seaborn for visualizations, and KaggleHub for dataset download.

Component	Implementation
Language	Python
Environment	Google Colab
Data handling	Pandas, NumPy
NLP representation	scikit-learn TfidfVectorizer
Classifier	scikit-learn LogisticRegression
Evaluation	precision, precision, recall, F1, ROC-AUC, confusion matrix
Audit	Pandas-based subgroup slicing
Explainability	Logistic Regression coefficient inspection

Table 3. Software components used in the setup.

The working framework is the Colab notebook itself, which loads the data, develops the model, checks it, runs the subgroup audit and prints the explainability output. No separate web or desktop user interface was put together for this assessment; so, the complete setup is meant to be kept up in the connected GitHub repository.

10. Findings and Evaluation

The model is found on 200 held-out inputs. Seeing that the task is, as a thing of fact, binary classification and the hate class is the primary mark of involvement, precision, recall and F1-score are accounted alongside precision and ROC-AUC.

Metric	Overall / Hate class	Value
Accuracy	Overall	0.695 (69.5%)
Precision	Hate	0.676 (67.6%)
Recall	Hate	0.575 (57.5%)
F1-score	Hate	0.621 (62.1%)
ROC-AUC	Overall	0.727
Precision	Non-Hate	0.706 (70.6%)
Recall	Non-Hate	0.788 (78.8%)
F1-score	Non-Hate	0.745 (74.5%)
Macro F1	Both classes	0.683 (68.3%)

Table 4. Test-set performance of the TF-IDF + Logistic Regression model.

What the confusion matrix includes is 89 true non-hate predictions, 24 false positives, 37 false negatives and 50 true hate predictions, and the model therefore is, basically, spotting 50 of 87 hate comments while losing 37. Precision is 69.5%, which is normally above the 56.5% majority-class baseline but besides suggests the straightforward baseline model has major room for refinement.

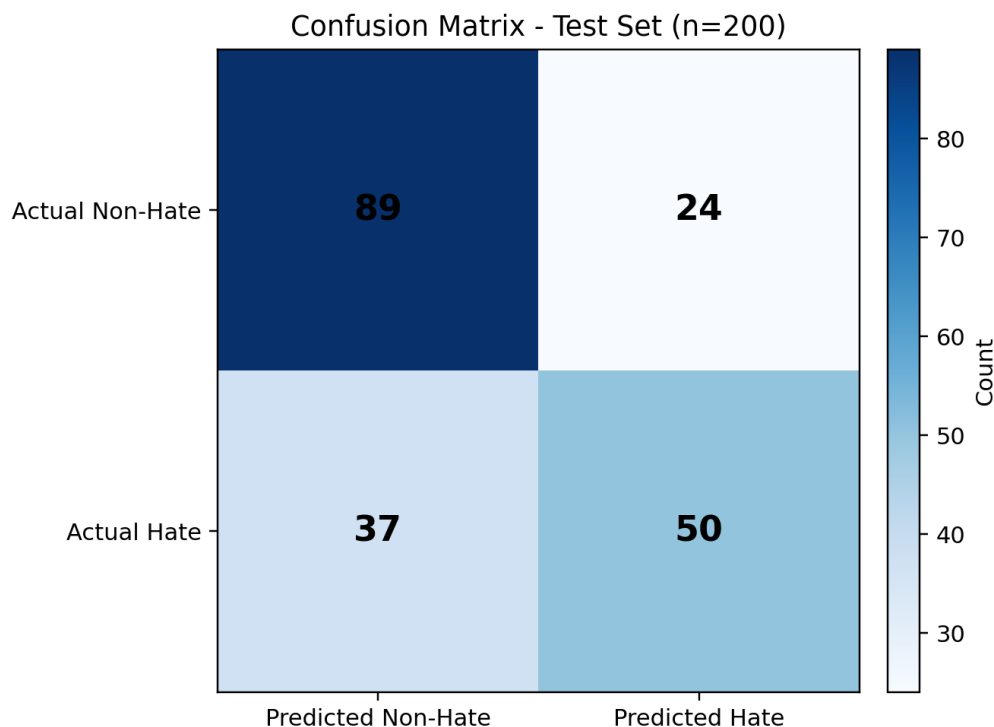


Figure 3. Confusion matrix for the held-out test set.

Actual \ Predicted	Non-Hate	Hate
Non-Hate	89	24
Hate	37	50

Table 5. Confusion matrix for the 200-comment test set.

Performance audit

Audit group	Samples	Accuracy	Hate Recall	Precision	F1
Short comments	104	68.3%	59.5%	61.0%	60.2%
Long comments	96	70.8%	55.6%	75.8%	64.1%
Profanity	31	67.7%	89.5%	68.0%	77.3%
No profanity	169	69.8%	48.5%	67.3%	56.2%

Table 6. Performance across two text subgroups/styles.

Relatively little changes in precision and recall are shown by the short-versus-long audit, even, though precision is higher for long comments. The subgroup audit is a text-style audit, not a demographic fairness audit. The larger difference looks in the profanity audit: hate recall is 89.5% when profanity is present but simply 48.5% when profanity is absent. What's more, 8 of 12 non-hate inputs including profanity are incorrectly flagged, suggesting that profanity may act as a shortcut feature.

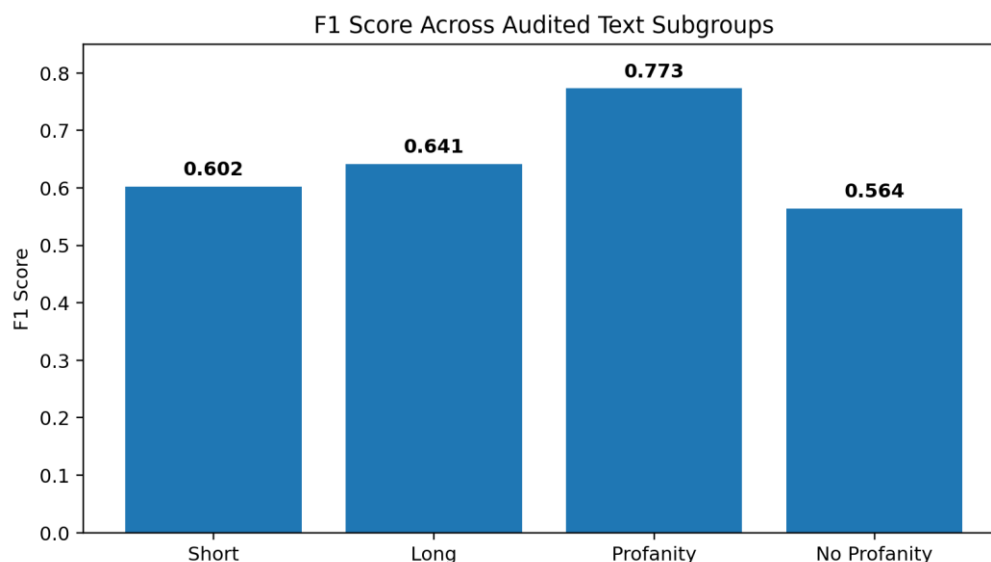


Figure 4. F1-score across the audited text subgroups.

11. Error Analysis / NLP Failure Analysis

The principal failure pattern is generally context dependence. The model is currently working from word and short-phrase associations instead than understanding the design of the full sentence. In a bit, clear offensive vocabulary can, as a thing of fact, increase the hate score even when the input isn't divided as hate in the dataset. At the same time, hate articulated without obvious profanity can be lost.

Representative demonstration cases

Text	Model output	Hate probability	Interpretation
"People from that group should leave"	Non-Hate	0.36	Shows a possible girl made by weak lexical indication despite exclusionary language.
"I disagree with your opinion"	Hate	0.57	Shows a false-positive habit, when the model overweights got lexical patterns.

Table 7. Qualitative demonstration examples used for error analysis; these aren't portion of the held-out test set.

The audit too indicates profanity-containing non-hate commentaries are often false positives; this behaviour is often coherent with the feature-level analysis, where strongly hate-leaning attributes are encompassing offensive vocabulary and identity-related terms. The model therefore complicates stating the presence of a word from the purpose or context in which that word is, say, employed.

12. Ethical Considerations and Shortcomings

Bias and fairness

The audit in this labor is emphasizing on text styles instead than demographic subgroups, because the binary ETHOS file doesn't offer a direct demographic credit for every example. The profanity audit uncovers a meaningful performance gap and indicates the model may acquire a shortcut association

between profanity and hate. Identity-related words can besides have strong positive coefficients, which is making a risk of false flags when groups are mainly tackled in neutral or non-hateful contexts.

Limitations

- The dataset is comprising just 998 inputs. What's more, the held-out test set includes 200 comments, so subgroup estimates can be noisy.
- The model loses a major proportion of hate comments; hate-class recall is typically 57.5%.
- TF-IDF + Logistic Regression doesn't clearly pattern setting, sarcasm, negation scope or conversation history.
- The assessment is English-only and may not generalize to multilingual, code-mixed or platform-specific language.
- The binary label is generally obtained from a 0.5 threshold on the original hate score, so edge cases may, more correctly, reflect annotation uncertainty.
- One train/test split is being employed by the task and doesn't run cross-validation or hyperparameter tuning.

This model should be employed merely as a first-pass moderation-support tool with human review for these grounds; it shouldn't be used as the sole basis for automatic removal, account bans or other punishing actions.

13. Conclusion

A complete NLP pipeline for binary hate-speech detection using the ETHOS dataset is successfully implemented by this labor; the pipeline cleans, stratified train/test splitting, TF-IDF feature extraction and Logistic Regression classification. 69.5% accuracy and a hate-class F1-score of 62.1% on the held-out test set are hit by the model, and the explainability analysis too is suggesting that the classifier rests heavily on particular lexical attributes. The subgroup audit generalizes the check beyond overall performance and spots a clear difference between inputs with and without profanity.

14. Future Scope

- Use character n-grams and compare substitute linear classifiers such as Linear SVM.
- Use cross-validation and a validation split for threshold selection and hyperparameter tuning.
- Expand the audit to encompass identity-term trades and other commanded text-style tests.
- Evaluate transformer-based frameworks such as BERT or RoBERTa to represent setting and phrase-level semantics.
- Increase dataset size and include more recent, multilingual and platform-diverse inputs.
- Adjust prediction thresholds for a moderation setting where recall may be ranked first over raw precision.

15. References

1. [1] ETHOS Hate Speech Dataset, Kaggle. "ETHOS Hate Speech Dataset." Available at: <https://www.kaggle.com/datasets/konradb/ethos-hate-speech-dataset>
2. [2] I. Mollas, Z. Chrysopoulou, S. Karlos, and G. Tsoumakas, "ETHOS: an Online Hate Speech Detection Dataset," arXiv:2006.08328, 2020. DOI: <https://doi.org/10.1007/s40747-021-00608-2>

3. [3] Scikit-learn Documentation, “TfidfVectorizer.” Available at: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
4. [4] Scikit-learn Documentation, “LogisticRegression.” Available at: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

16. GitHub Repository

GitHub Repository Link: To be added before final submission.

The store should comprise the last Jupyter notebook, dataset or a consistent dataset-download measure, README.md, and the supporting report/model-card artifacts needed for assessment.

17. Team Contribution

The contribution allocation below is, in practice, a structured draft grounded on the project deliverables and should be backed up by the squad before final submission.

Team Member	PRN	Specific Contribution
Jashan Chaudhary	25030422054	NLP pipeline implementation, model training/evaluation, project coordination.
Sanjeev Katragadda	25030422059	Dataset preparation, subgroup performance audit, findings construal.
Agustya Gupta	25030422011	Explainability analysis, model-card content, shortcomings and risk analysis.
Ansh Sahani	25030422016	Describe documentation, presentation preparation, terminal project organization.

Table 8. Team contribution breakdown.

18. Annexure (if needed)

Annexure A, One-page model card

The one-page model card is passed in as a separate artifact; it recaps the model, dataset, overall performance, two subgroup audits, explainability findings, shortcomings and responsible-use recommendations.

Annexure B, Key explainability output

Feature	Coefficient	Interpretation
ass	+1.4239	Strong hate-leaning attribute
kill	+1.1557	Hate-leaning trait
gays	+1.1343	Identity-related hate-leaning characteristic
fuck	+1.1036	Profanity-related hate-leaning attribute
stupid	+1.0709	Negative/offensive lexical signal
love	-1.1532	Non-hate-leaning attribute
can	-1.0982	Non-hate-leaning attribute
me	-1.0592	Non-hate-leaning attribute
the	-1.0219	Non-hate-leaning attribute
am	-0.9393	Non-hate-leaning attribute

Table 9. Example top Logistic Regression coefficients used for explainability.

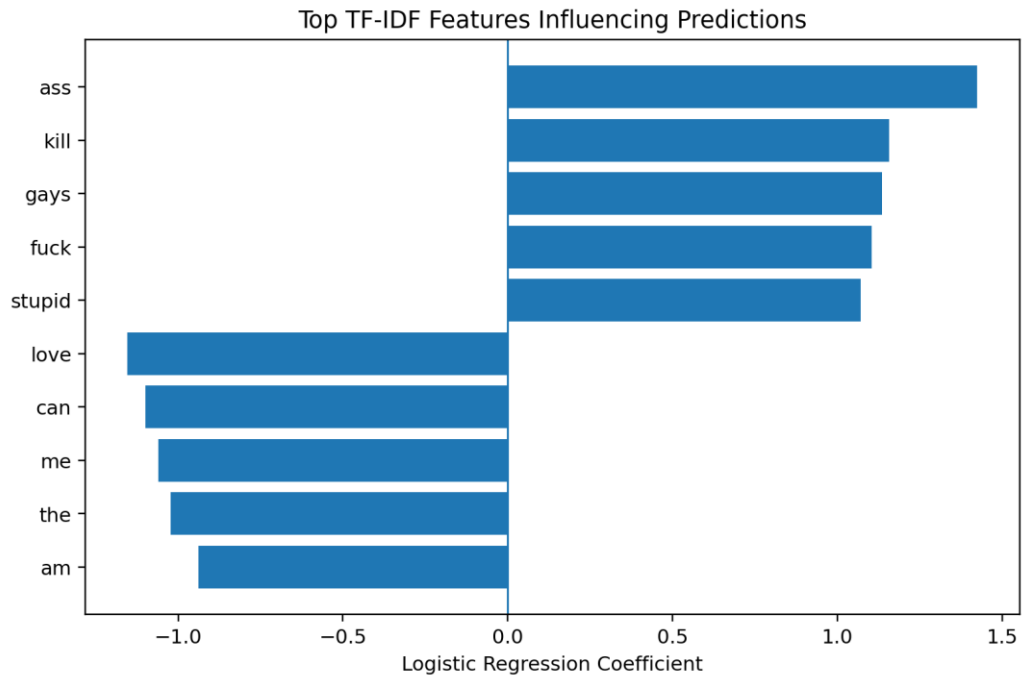


Figure 5. Top TF-IDF attributes affecting hate/non-hate predictions.